CSE/ISyE 6740
Computational Data Analysis

LLM Training and Finetuning

11/11/2026

Kai Wang, Assistant Professor in Computational Science and Engineering
kwang692@gatech.edu

Georgia Tech

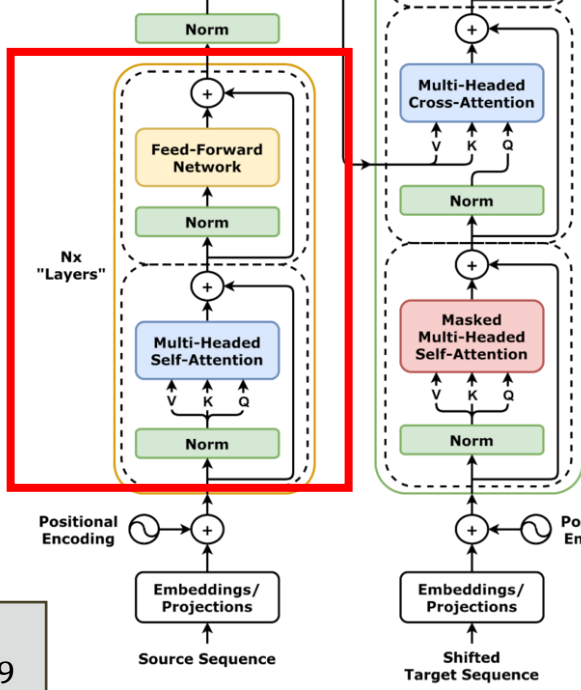
Outline

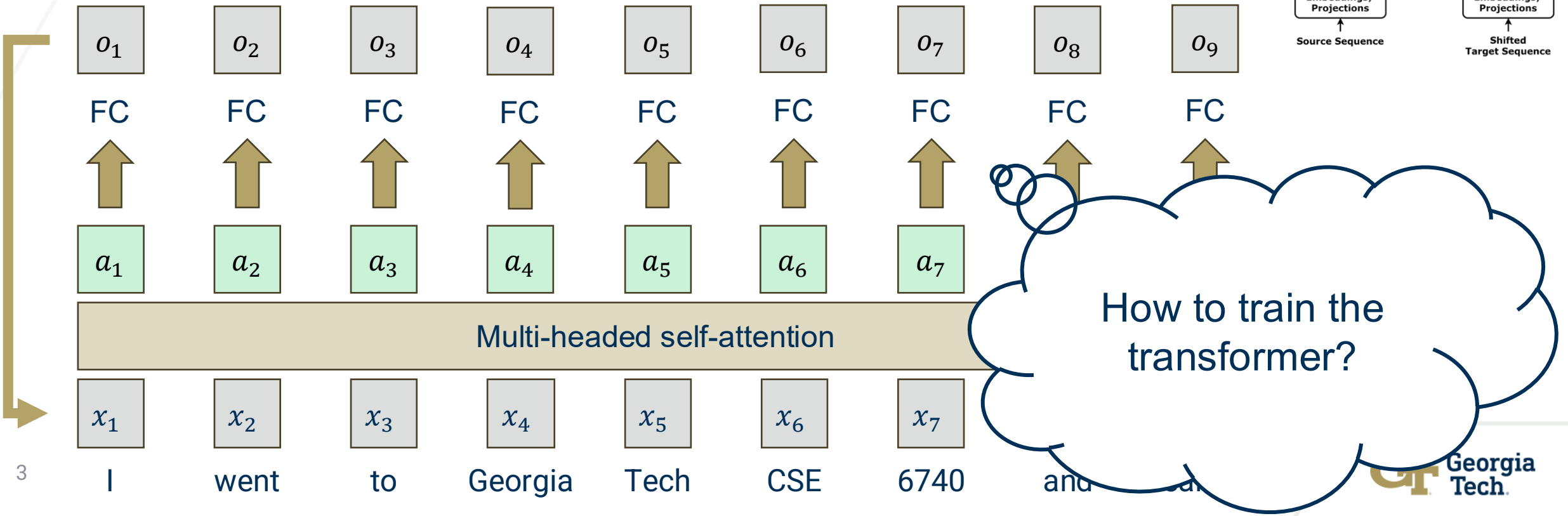
- **Advanced topics**
  - Transformer
    - Pre-training
    - Fine-tuning
    - Other LLM techniques

Georgia Tech

Transformer

- All these operators compose the multi-headed self-attention part.
- Repeat for N iterations





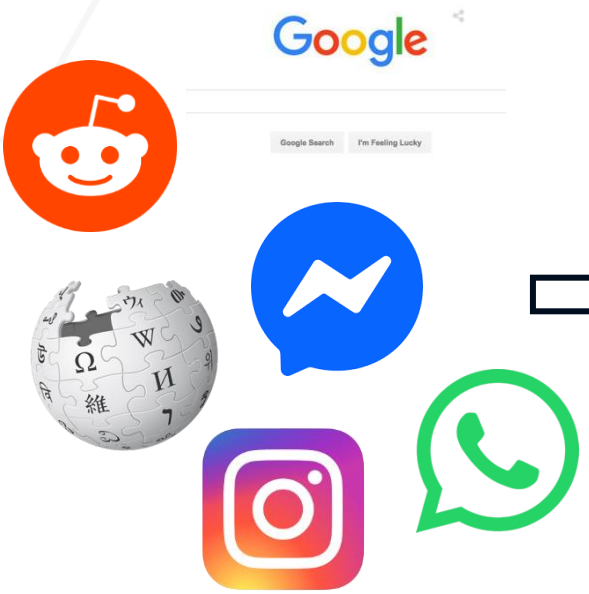
Georgia Tech

LLM Pre-training

Georgia Tech

Training Transformer

- We have a transformer model, e.g., GPT. Now, let's prepare the data to train the model





```python
class GPT(nn.Module):
    """ GPT Language Model """

    @staticmethod
    def get_default_config(): ...
        return C

    def __init__(self, config): ...
        print("number of parameters: %.2fM" % (n_params/1e6,))

    def _init_weights(self, module): ...
            torch.nn.init.ones_(module.weight)

    @classmethod
    def from_pretrained(cls, model_type): ...
        return model

    def configure_optimizers(self, train_config): ...
        return optimizer

    def forward(self, idx, targets=None):
        device = idx.device
        b, t = idx.size()
        assert t <= self.block_size, f"Cannot forward sequence of length {t}, block size is only {self.block_size}"
        pos = torch.arange(0, t, dtype=torch.long, device=device).unsqueeze(0) # shape (1, t)

        # forward the GPT model itself
        tok_emb = self.transformer.wte(idx) # token embeddings of shape (b, t, n_embd)
        pos_emb = self.transformer.wpe(pos) # position embeddings of shape (1, t, n_embd)
        x = self.transformer.drop(tok_emb + pos_emb)
        for block in self.transformer.h:
            x = block(x)
        x = self.transformer.ln_f(x)
        logits = self.lm_head(x)

        # if we are given some desired targets also calculate the loss
        loss = None
        if targets is not None:
            loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1), ignore_index=-1)

        return logits, loss

    @torch.no_grad()
    def generate(self, idx, max_new_tokens, temperature=1.0, do_sample=False, top_k=None): ...
        return idx
```

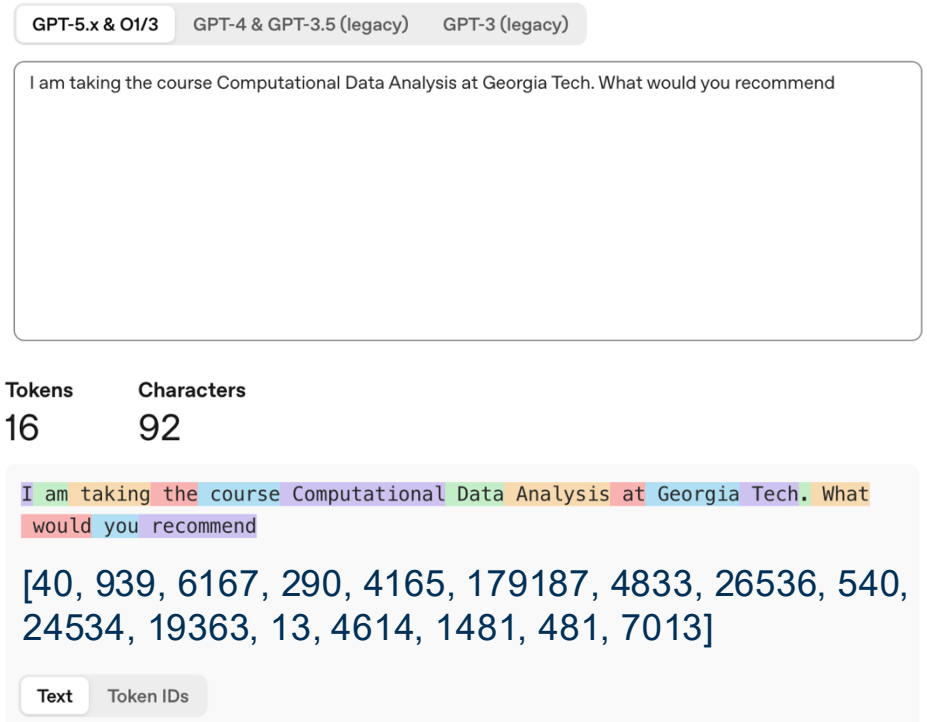
https://github.com/karpathy/minGPT/blob/master/mingpt/model.py

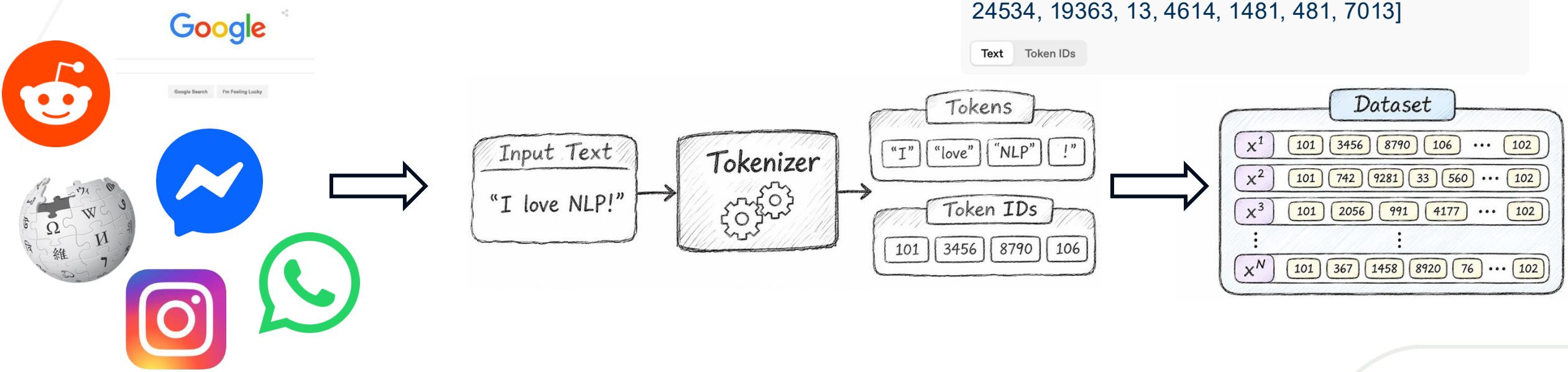
Georgia Tech

Training Transformer

- Prepare a large corpus of language data
- Feed them into a tokenizer
- Construct a dataset

https://platform.openai.com/tokenizer





Georgia Tech

Training Transformer

- How to compute the loss function?

$inputs \in \mathbb{R}^{B \times L}$

```
inputs = torch.tensor([[16833, 3626, 6100],   # ["every effort moves",
                       [40,    1107, 588]])   #  "I really like"]
```

```
targets = torch.tensor([[3626, 6100, 345  ],  # [" effort moves you",
                        [1107,  588, 11311]]) #  " really like chocolate"]
```

$logits = model(inputs) \in \mathbb{R}^{B \times L \times d_{vocab}}$

$targets \in \mathbb{R}^{B \times L}$

```
# Logits have shape (batch_size, num_tokens, vocab_size)
print("Logits shape:", logits.shape)

# Targets have shape (batch_size, num_tokens)
print("Targets shape:", targets.shape)
```

```
Logits shape: torch.Size([2, 3, 50257])
Targets shape: torch.Size([2, 3])
```

Flatten and compute the loss
*loss = **cross_entropy**(logits, targets)*
loss.backward()

probs = softmax(logits) $\in \mathbb{R}^{B \times L \times d_{vocab}}$
cross_entropy = $-\sum_i y_i \log \hat{y}_i$

Georgia Tech

Training Transformer

- How to compute the loss function?

What is the computation cost for **RNN/LSTM** and **Transformer**?

$inputs \in \mathbb{R}^{B \times L}$

```
inputs = torch.tensor([[16833, 3626, 6100],   # ["every effort moves",
                       [40,    1107, 588]])   #  "I really like"]
```

$logits = model(inputs) \in \mathbb{R}^{B \times L \times d_{vocab}}$

```
targets = torch.tensor([[3626, 6100, 345  ],  # [" effort moves you",
                        [1107,  588, 11311]]) #  " really like chocolate"]
```

$targets \in \mathbb{R}^{B \times L}$

```
# Logits have shape (batch_size, num_tokens, vocab_size)
print("Logits shape:", logits.shape)

# Targets have shape (batch_size, num_tokens)
print("Targets shape:", targets.shape)
```

```
Logits shape: torch.Size([2, 3, 50257])
Targets shape: torch.Size([2, 3])
```

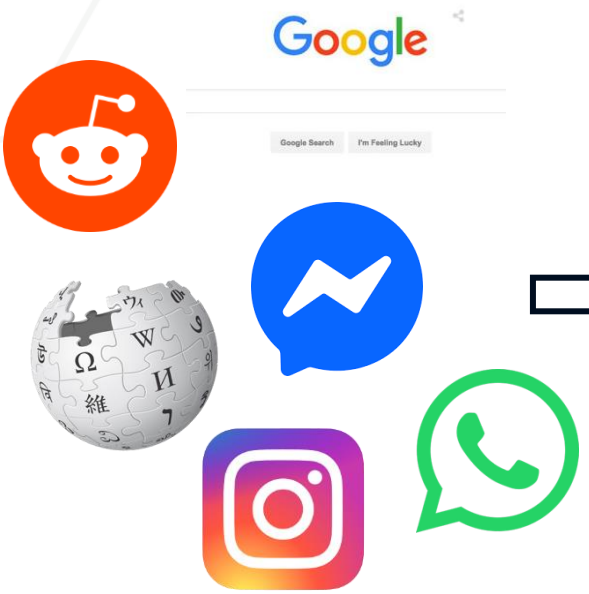
Flatten and compute the loss
*loss = **cross_entropy**(logits, targets)*
loss.backward()

probs = softmax(logits) $\in \mathbb{R}^{B \times L \times d_{vocab}}$
cross_entropy = $-\sum_i y_i \log \hat{y}_i$

Georgia Tech

Training Transformer

- After the Transformer is trained, we can now generate tokens from it.





```python
class GPT(nn.Module):

    @torch.no_grad()
    def generate(self, idx, max_new_tokens, temperature=1.0, do_sample=False, top_k=None):
        """
        Take a conditioning sequence of indices idx (LongTensor of shape (b,t)) and complete
        the sequence max_new_tokens times, feeding the predictions back into the model each time.
        Most likely you'll want to make sure to be in model.eval() mode of operation for this.
        """
        for _ in range(max_new_tokens):
            # if the sequence context is growing too long we must crop it at block_size
            idx_cond = idx if idx.size(1) <= self.block_size else idx[:, -self.block_size:]
            # forward the model to get the logits for the index in the sequence
            logits, _ = self(idx_cond)
            # pluck the logits at the final step and scale by desired temperature
            logits = logits[:, -1, :] / temperature
            # optionally crop the logits to only the top k options
            if top_k is not None:
                v, _ = torch.topk(logits, top_k)
                logits[logits < v[:, [-1]]] = -float('Inf')
            # apply softmax to convert logits to (normalized) probabilities
            probs = F.softmax(logits, dim=-1)
            # either sample from the distribution or take the most likely element
            if do_sample:
                idx_next = torch.multinomial(probs, num_samples=1)
            else:
                _, idx_next = torch.topk(probs, k=1, dim=-1)
            # append sampled index to the running sequence and continue
            idx = torch.cat((idx, idx_next), dim=1)

        return idx
```

Get the logits

Convert to probabilities

Sampling

Append and repeat

https://github.com/karpathy/minGPT/blob/master/mingpt/model.py

Georgia Tech

Questions

- What is the computation cost of **feeding a sequence** of length L into the Transformer model?

- What is the computation cost of **generating a sequence** of length L from a Transformer model?

Georgia Tech

Cost of Generation

Pricing

Flagship models

Speed: Standard | Flex/Batch | Fast | Ultrafast

Context length: Short | Long

Prices per 1M tokens.

Model	Input	Output	Prompt caching: Cached input	Prompt caching: 30m write	Prompt caching: 12h write
gpt-6-astra	$10.00	$50.00	$1.00	$12.50	$25.00
gpt-6.1-sol	$2.00	$10.00	$0.10	$2.50	$5.00
gpt-6.1-terra	$1.00	$5.00	$0.05	$1.25	$2.50
gpt-6-luna	$0.10	$0.50	$0.01	$0.125	$0.25

All models



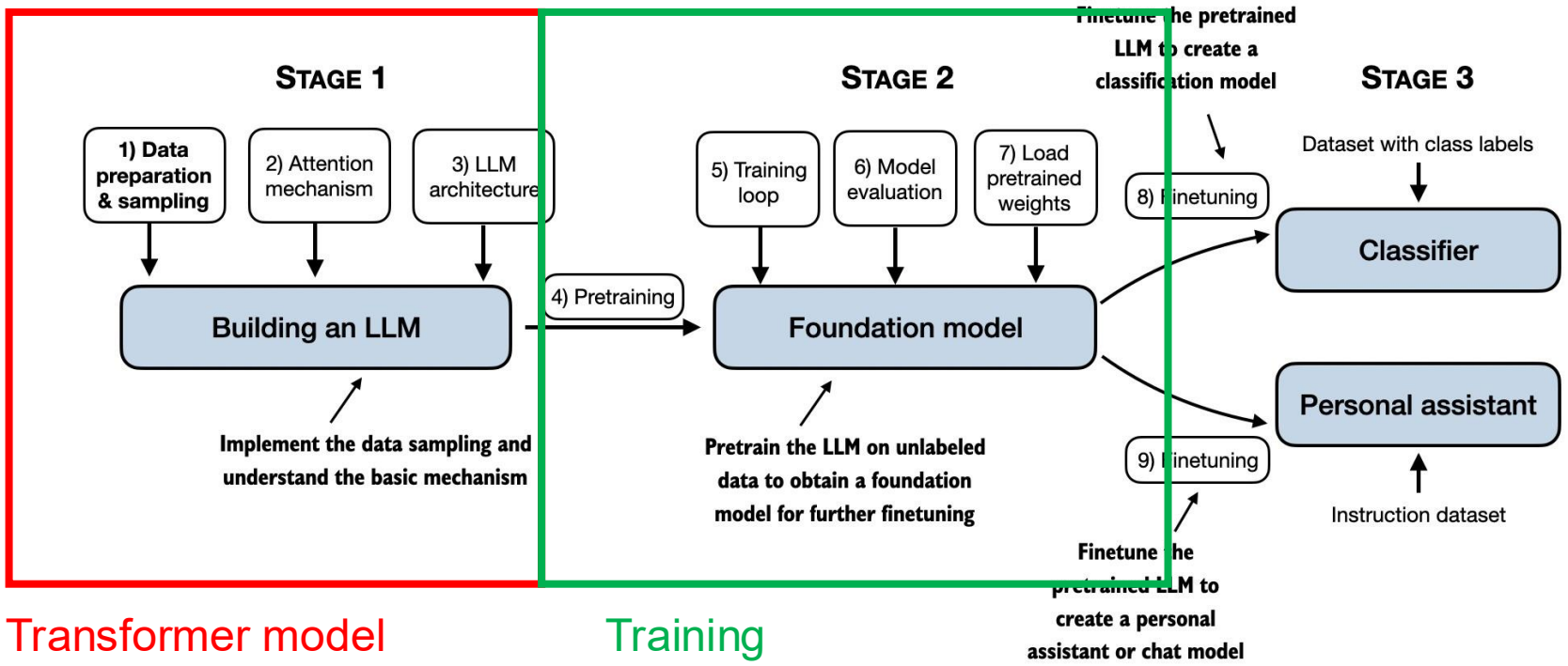


Model pricing

The following table shows pricing for all Claude models:

Model	Base tokens		Prompt caching		
Name	Input	Output	5m writes	1h writes	Hits and refreshes
Claude Fable 5.2 For demanding reasoning and long-horizon agentic work	$8 / MTok	$40 / MTok	$10 / MTok	$16 / MTok	$0.20 / MTok
Claude Opus 5.5 For long-running agentic coding and knowledge work	$4 / MTok	$20 / MTok	$5 / MTok	$8 / MTok	$0.20 / MTok
Claude Sonnet 5.5 The best combination of speed and intelligence	$2 / MTok	$10 / MTok	$2.50 / MTok	$4 / MTok	$0.10 / MTok
Claude Haiku 5.5 For high-volume, latency-sensitive tasks such as classification, extraction, and routing	$0.10 / MTok for prompts up to 100,000 tokens	$0.50 / MTok	$0.125 / MTok	$0.20 / MTok	$0.01 / MTok
	$0.50 / MTok for prompts over 100,000 tokens	$2.50 / MTok	$0.625 / MTok	$1 / MTok	$0.05 / MTok

How to Train LLMs?



https://github.com/rasbt/LLMs-from-scratch

Ok so are you ready to train your LLMs?

- We just need some GPUs and some RAM! Let's estimate how much $$ we need!

- **Beam** from Reflection AI
  - **501B** parameters (~2,004GB parameters, ~8,000GB to run gradient descent)
  - 288GB per NVidia B300 GPU
  - Roughly 28 GPUs needed

B300 GPUs	1 trillion tokens	10 trillion tokens
32	~75 days	~752 days
64	~38 days	~376 days
128	~19 days	~188 days
256	~9 days	~94 days
512	~5 days	~47 days

(estimated by ChatGPT)



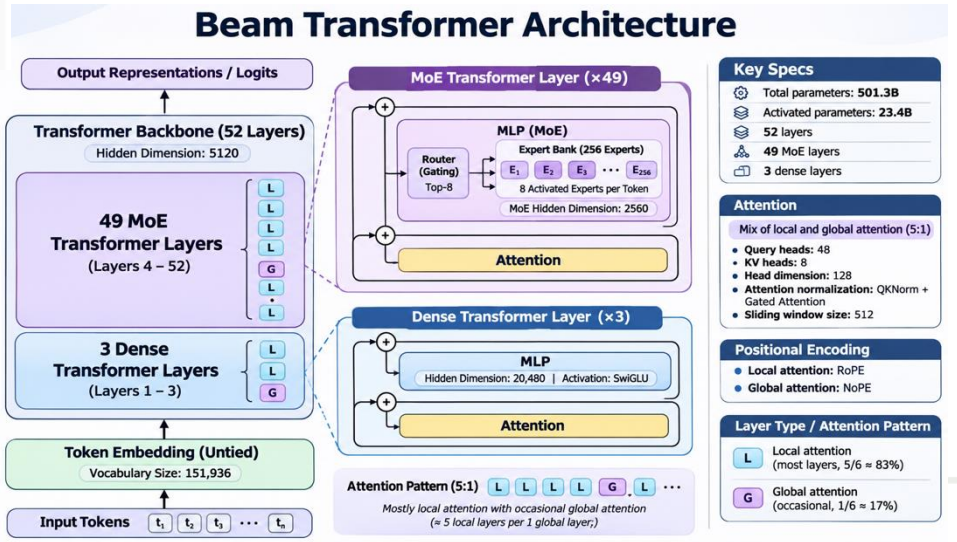
Rent NVIDIA B300 288GB 8-GPU Cloud ...

$445,008.00

tokens ~ 20x #parameters

Configuration	Beam
Number of Layers	52
Total Parameters	501.3B
Activated Parameters	23.4B
Hidden Dimension	5120
MoE Hidden Dimension	2560
Total Experts per Layer	256
Activated Experts per Token	8
Number of MoE Layers	49
MLP Hidden Dimension	20480
Number of Dense Layers	3
MLP/MoE Activation Function	SwiGLU
Query Heads	48
KV Heads	8
Head Dimension	128
Attention Mechanism	Mix of local and global (5:1)
Attention Normalization	QKNorm + Gated Attention
Sliding Window Size	512
Positional Encoding	Local attention: RoPE Global attention: NoPE
Embedding	Untied
Vocabulary Size	151936

Table 1: Beam configuration and hyperparameters.



https://reflection.ai/documents/beam-model-tech-report

Ok so are you ready to train your LLMs?

- Maybe let's try a smaller model…

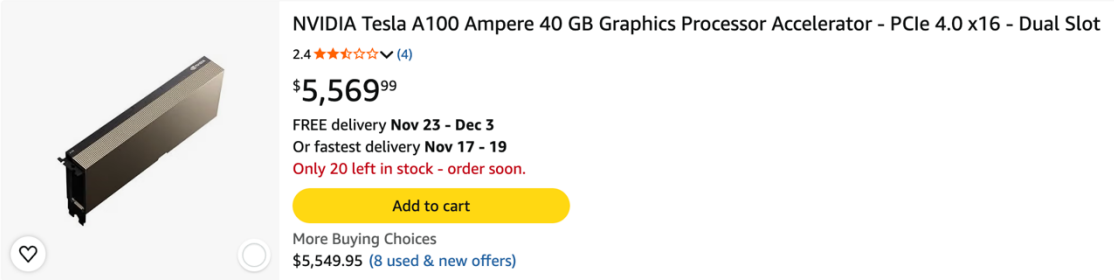
- **Qwen8B** from Tongyi Qianwen
  - **8B** parameters (~32GB parameters, ~128GB to run gradient descent)
  - 40/80GB per NVidia A100 GPU
  - Roughly 3~4 GPUs needed!!

Qwen3-8B Pretraining Time — NVIDIA A100 40GB

A100 40GB GPUs	16B tokens (2×)	160B tokens (20×)
8	~11.9 days	~119 days
16	~5.9 days	~59 days
32	~3.0 days	~30 days
64	~1.5 days	~15 days
128	~17.8 hours	~7.4 days
256	~8.9 hours	~3.7 days
512	~4.5 hours	~1.9 days

(estimated by ChatGPT)

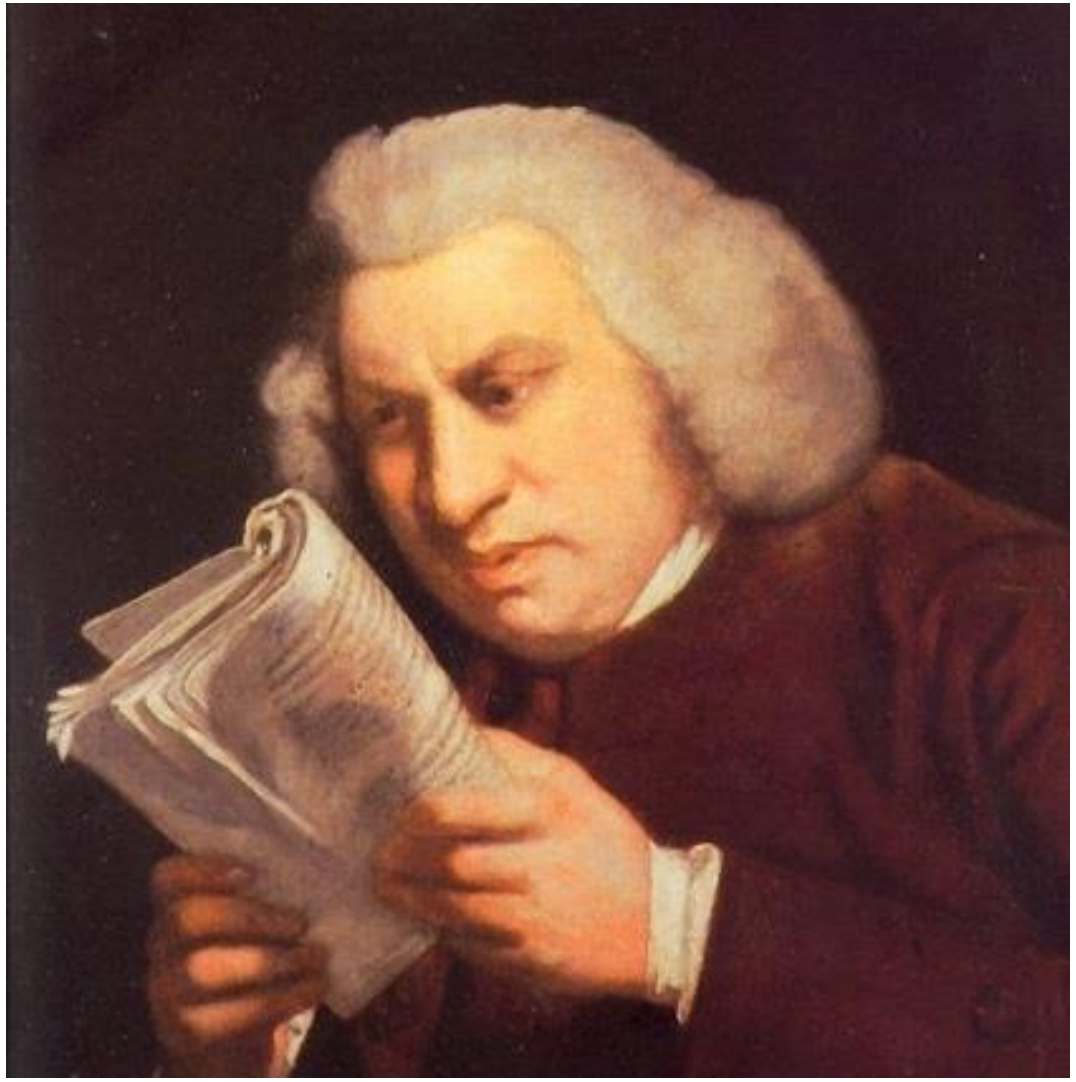
tokens ~ 20x #parameters



GPT renting price (GT PACE):

- ~$0.5 per A100 GPU hour
- We need ~22,000 GPU hours

???



Georgia Tech

LLM Finetuning

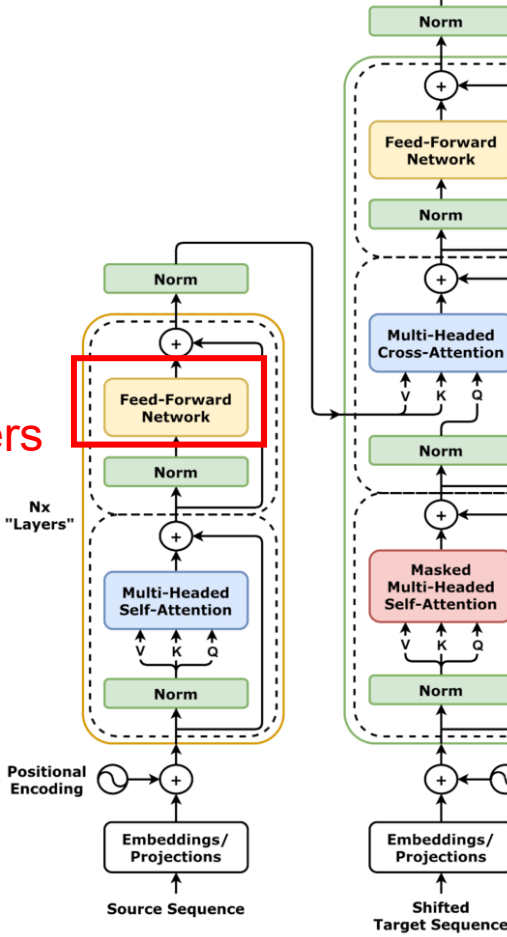
Georgia Tech

Matrix Multiplication in Transformer

- We have too many parameters in matrix multiplication!

  - Query parameters $W^q \in \mathbb{R}^{d \times d_k}$
  - Key parameters $W^k \in \mathbb{R}^{d \times d_k}$
  - Value parameters $W^v \in \mathbb{R}^{d \times d_v}$

Fully connected layers



$x_i \times W^v = v_i$

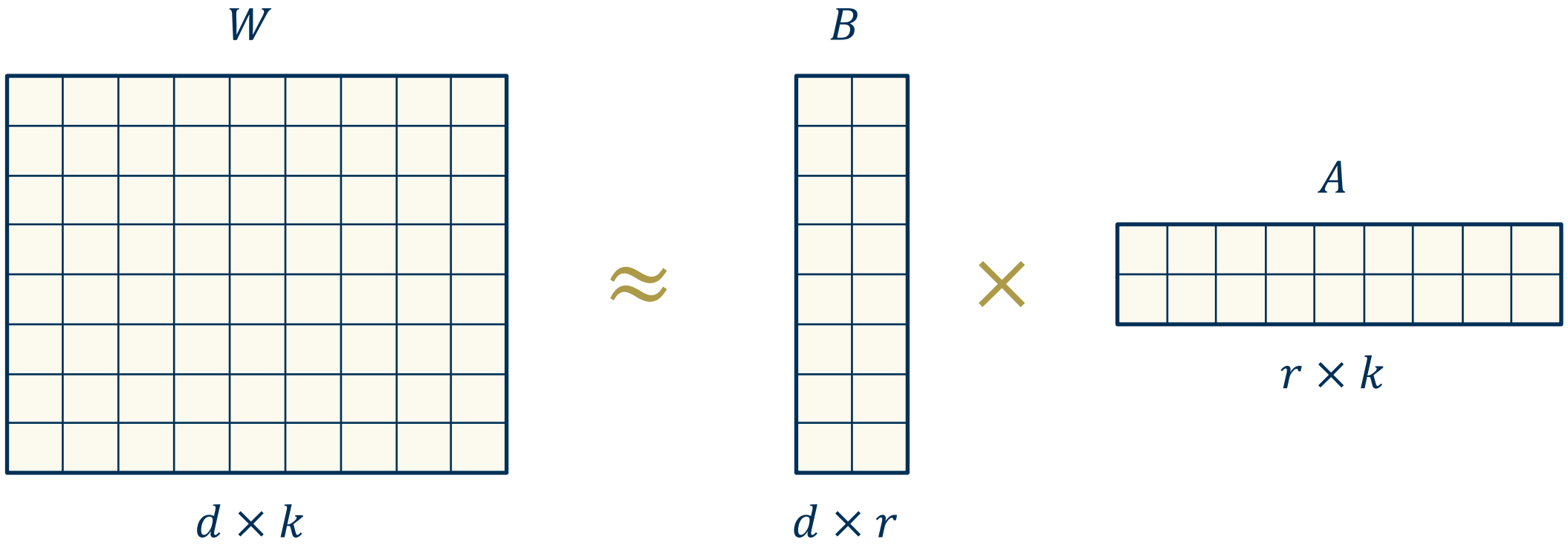
If we can simplify/accelerate matrix multiplication, then we can save cost!

I	went	to	Georgia	Tech	CSE	6740	and	learned
			v_4	v_5	v_6	v_7	v_8	v_9
				k_5	k_6	k_7	k_8	k_9
				q_5	q_6	q_7	q_8	q_9

Georgia Tech

Low-Rank Approximation

- If the matrix W is low-rank, then we can approximate it by two shallow matrices



But this only works for "low-rank" matrix W

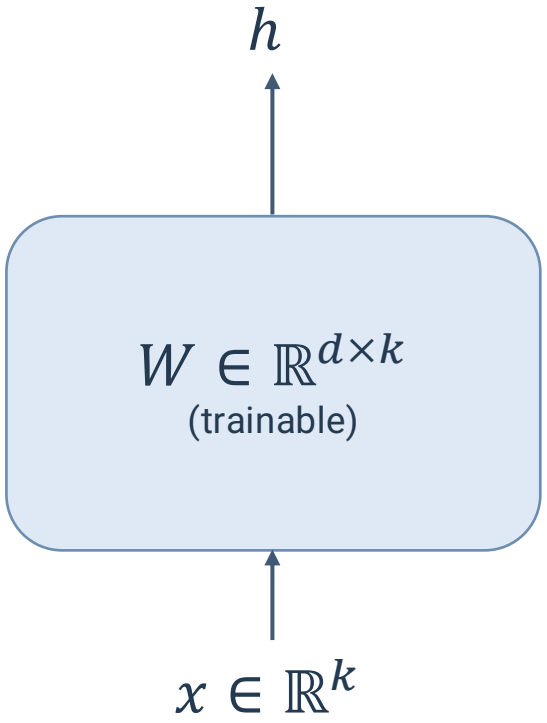
- It doesn't work for most of the NN parameters

Georgia Tech

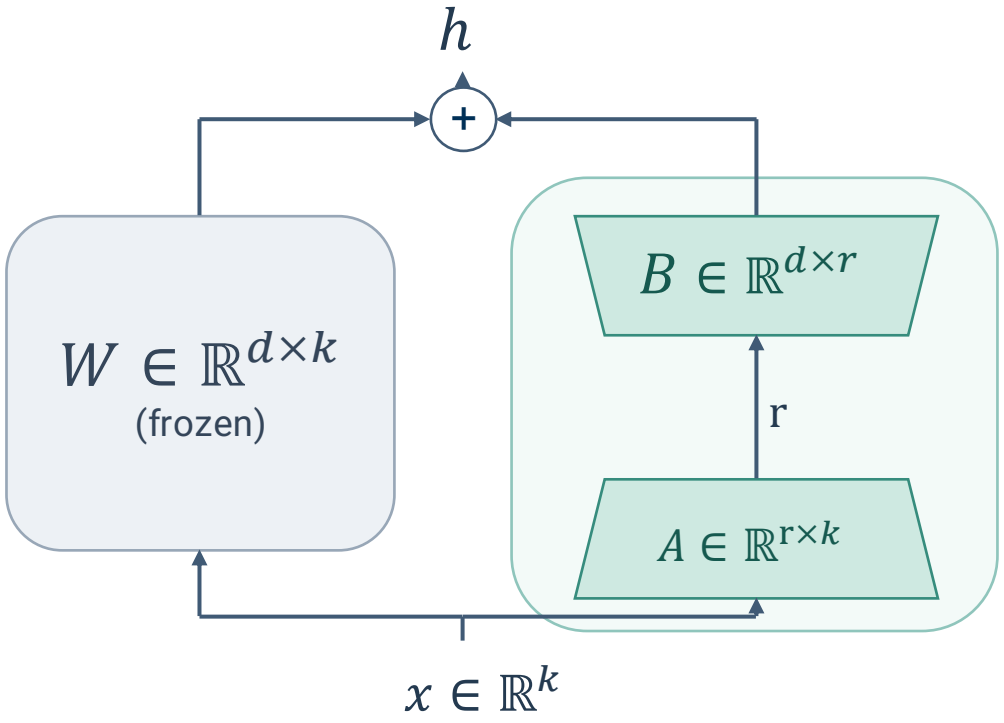
Low-Rank Adaptation (LoRA)

Instead of hoping the weight matrix W to be low-rank, we expect the “change” to be low-rank

Standard Fine-Tuning



LoRA: Low-Rank Adaptation

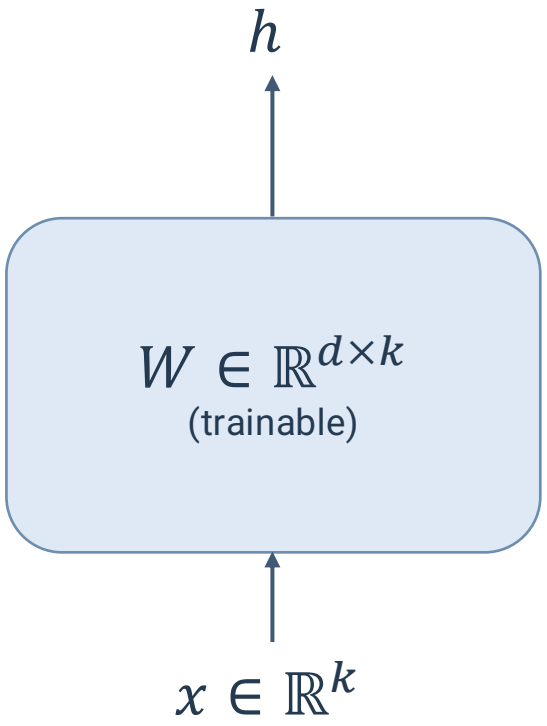


$$\Delta W = BA \quad (r \ll \min(d, k))$$

[LORA: Low-Rank Adaption of Large Language Models. Hu et al. 2021]

Low-Rank Adaptation (LoRA)

Standard Fine-Tuning

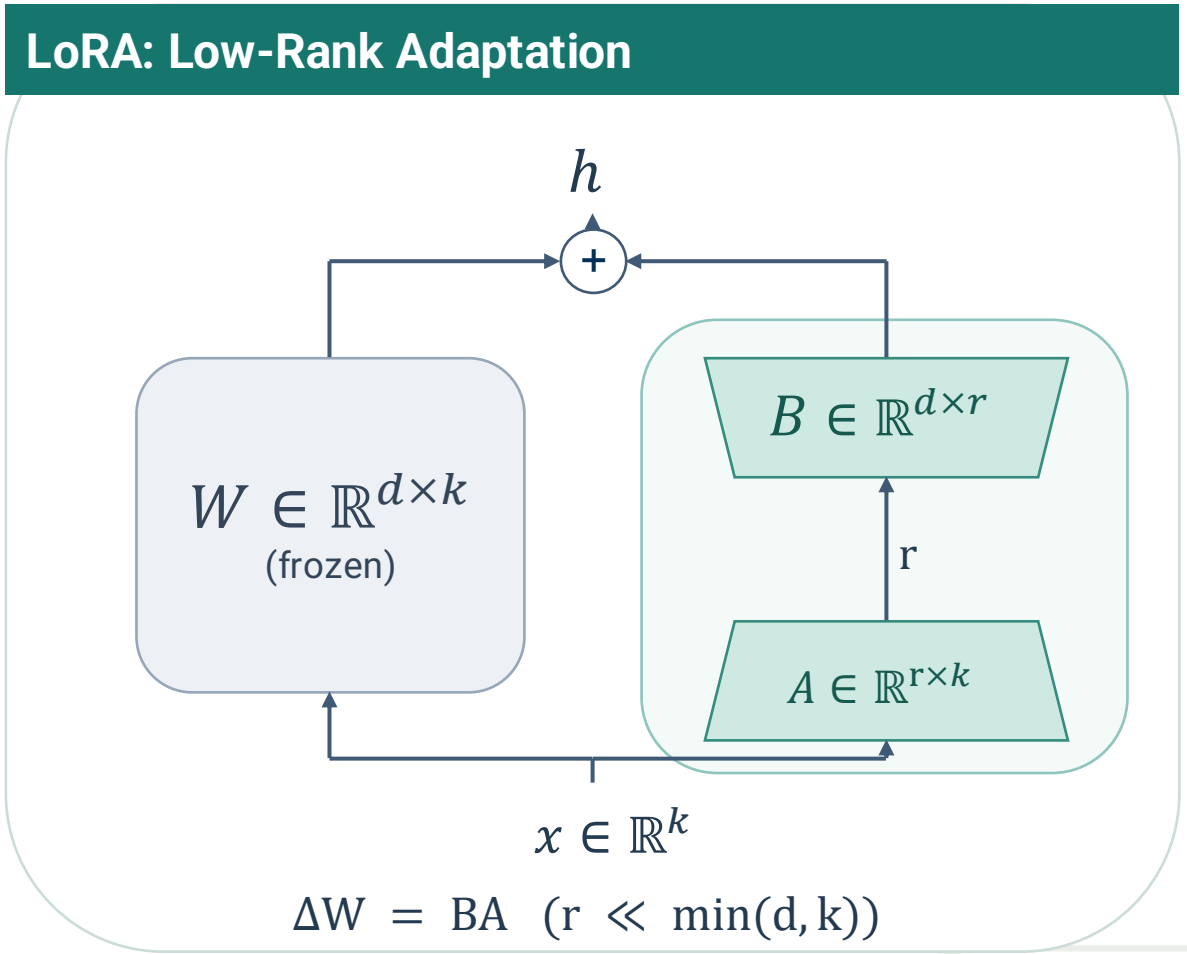


- What is the inference compute/memory cost?
- What is the training compute/memory cost?

Georgia Tech

Low-Rank Adaptation (LoRA)

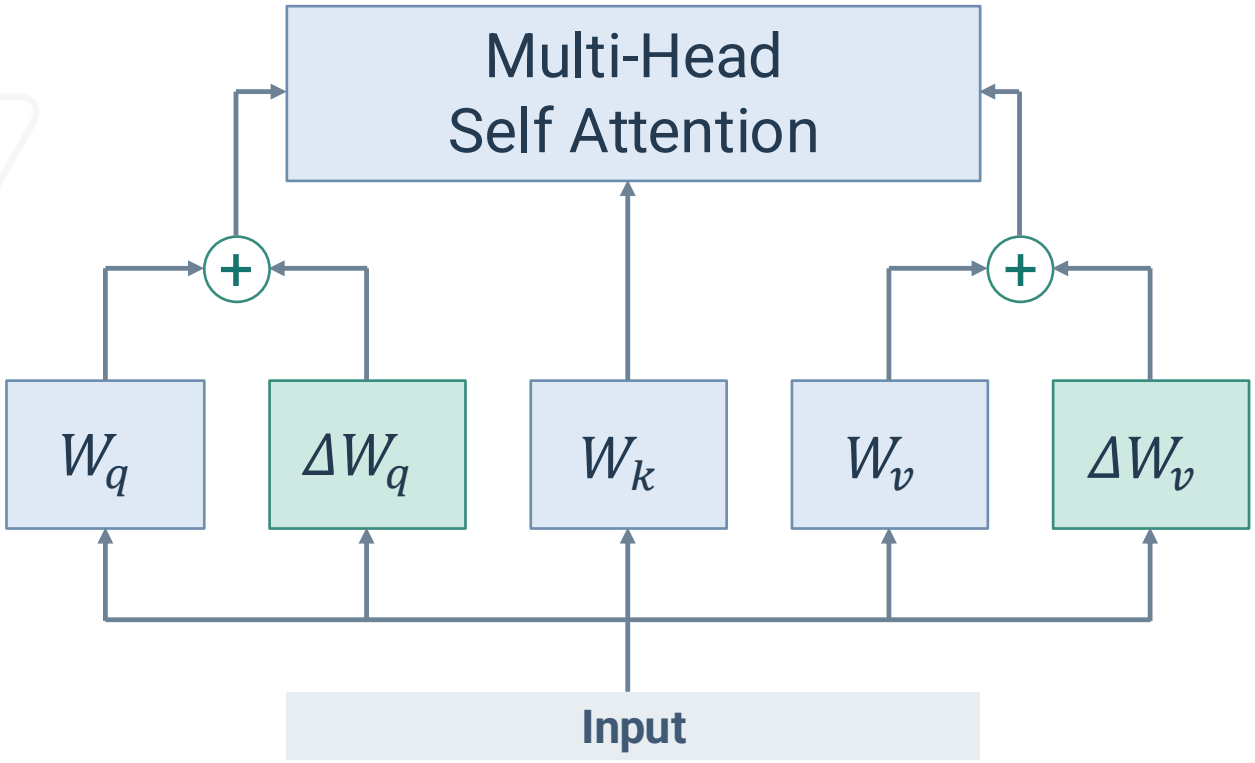
- What is the inference compute/memory cost?
- What is the training compute/memory cost?



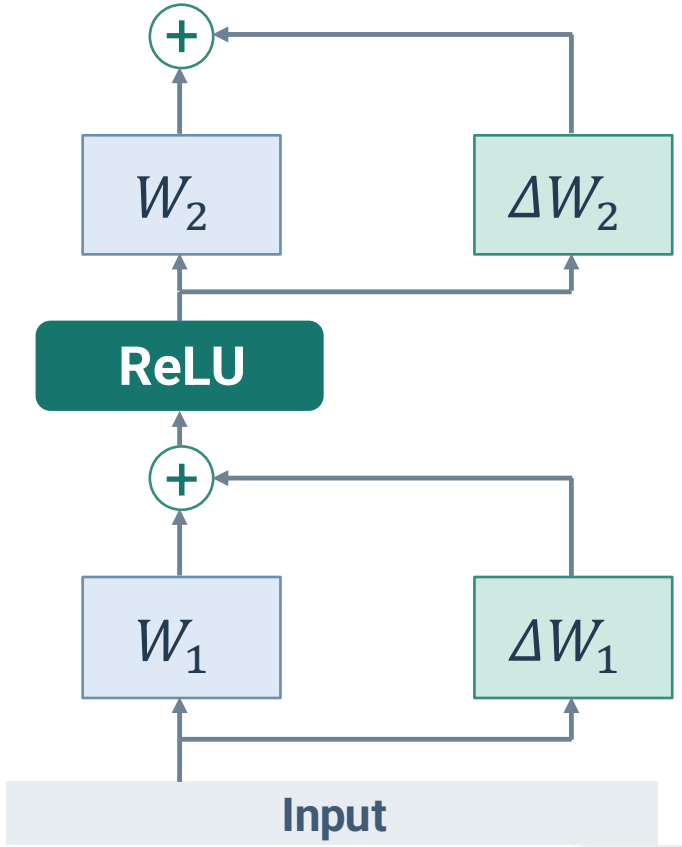
Georgia Tech

Transformer with LoRA

Apply LoRA to Attention



Apply LoRA to MLP layer



Georgia Tech

Memory Comparison of Transformer with LoRA

Model	Fine-tuning	Frozen weights	Trainable weights	Adam (3× trainable weights)	Total memory
Qwen3-8B	Full	—	32 GB	96 GB	**128 GB**
Qwen3-8B	LoRA	32 GB	0.03 GB	0.09 GB	**32.12 GB**
Beam 501.3B	Full	—	2,005.2 GB	6,015.6 GB	**8,020.8 GB**
Beam 501.3B	LoRA	2,005.2 GB	0.06 GB	0.17 GB	**2,005.4 GB**

Georgia Tech

Parameter Efficient Fine Tuning (PEFT)

- LoRA can be automatically integrated into your transformer model using the PEFT library.
- But you still need sufficient memory/compute to host the base model.

```python
import torch
from transformers import AutoModelForCausalLM
from peft import LoraConfig, TaskType, get_peft_model

device = torch.accelerator.current_accelerator().type if hasattr(torch, "accelerator") else "cu
model_id = "Qwen/Qwen2.5-3B-Instruct"
model = AutoModelForCausalLM.from_pretrained(model_id, device_map=device)
peft_config = LoraConfig(
    r=16,
    lora_alpha=32,
    task_type=TaskType.CAUSAL_LM,
    # target_modules=["q_proj", "v_proj", ...]  # optionally indicate target modules
)
model = get_peft_model(model, peft_config)
model.print_trainable_parameters()
# prints: trainable params: 3,686,400 || all params: 3,089,625,088 || trainable%: 0.1193

# now perform training on your dataset, e.g. using transformers Trainer, then save the model
model.save_pretrained("qwen2.5-3b-lora")
```

Specify the LoRA parameter. PEFT handles the rest for you.

Georgia Tech

Other LLM Techniques

Georgia Tech

LLM Quantization

Represent model weights and activations with lower-precision numbers (e.g., FP32/FP16 → INT8/INT4) to reduce memory and computation.

High-precision (FP16 / FP32)

$$W \in \mathbb{R}^{m \times n}$$

16/32 bits

0.42	−1.37	0.90	2.68
−0.91	0.31	1.52	−0.47
1.27	−2.34	0.63	0.15
−0.28	1.96	−1.08	0.44

Illustrative weight matrix

Quantization

Map to a lower-precision integer with a scale (and offset)

Low-precision (INT4 / INT8)

$$Q(W) \in \mathbb{Z}^{m \times n}$$

4/8 bits · integer example: INT8

2	−7	5	13
−5	2	7	−2
6	−12	3	1
−1	10	−6	2

$W \approx s \cdot Q(W)$ s: scale (e.g., 0.21)

Benefits

Lower memory

e.g., ~4x with INT4

Faster computation

e.g., higher throughput on supported GPUs

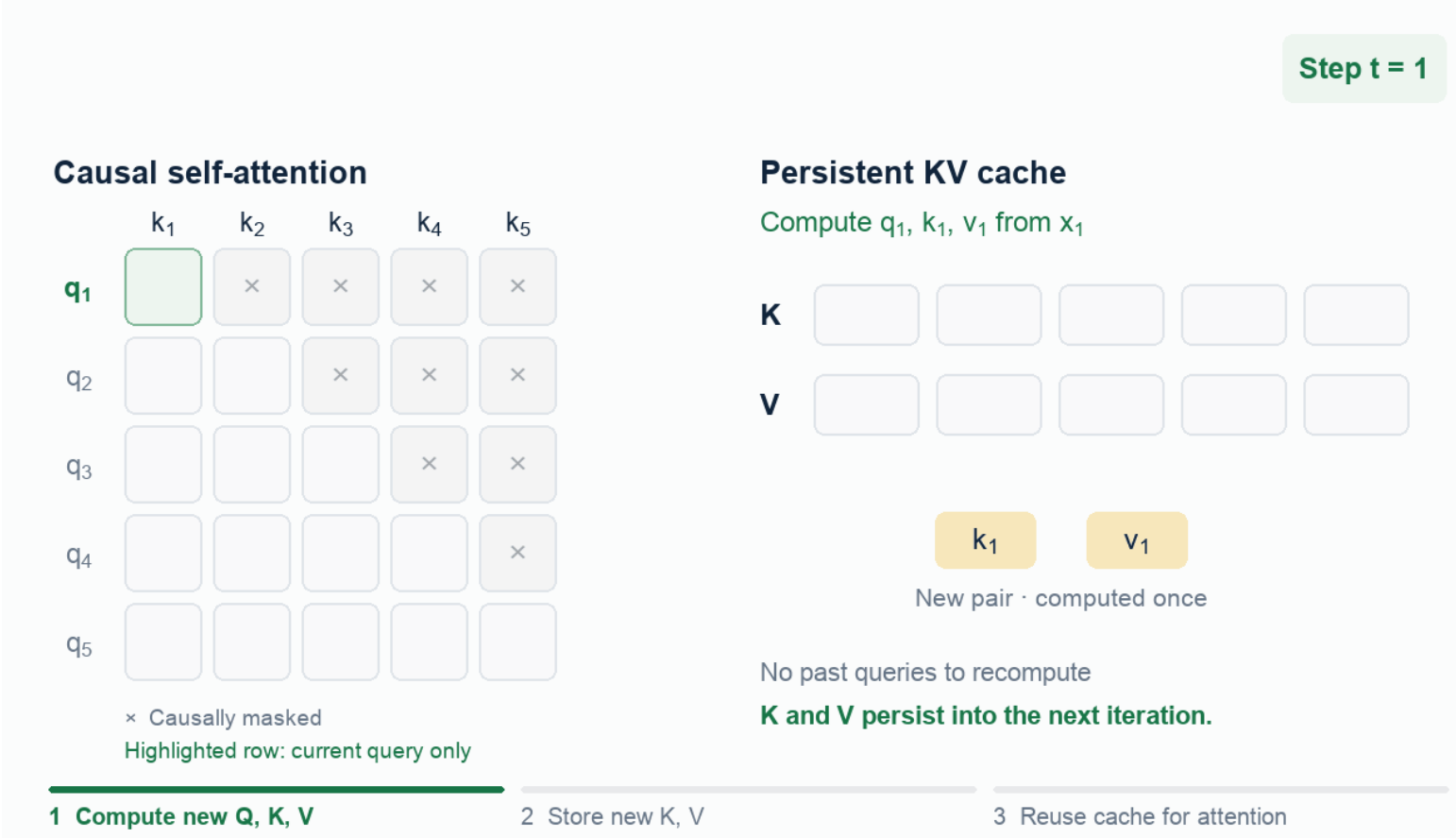
Enable larger models

or lower deployment cost

Georgia Tech

KV Cache

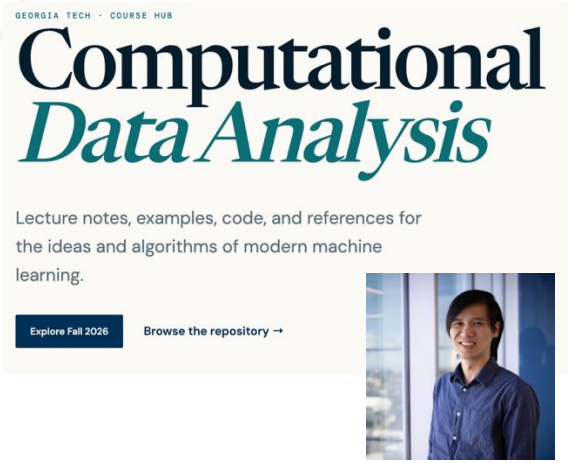
- Leverage the property of the **“causal attention mechanism”**.
- Store the keys and values
- At token t, we just compute the new key k_t, value v_t, and query q_t
- Computation becomes $O(t)$ at token t



Georgia Tech

Supervised Fine-Tuning

- If you want to fine-tune the base model to a specific dataset, then you can do supervised fine-tuning (SFT).
  - Just feed the datasets to compute cross entropy loss to train the LoRA.
  - Or if you have any other loss function, just use it to backprop and train the LoRA.



Datasets



Base model



Finetuned model

Georgia Tech

Questions

- Why don't we just directly replace the weight matrix $W \in \mathbb{R}^{d \times k}$ by $BA \approx W$ with low-rank $B \in \mathbb{R}^{d \times \mathrm{r}}$ and $A \in \mathbb{R}^{\mathrm{r} \times \mathrm{k}}$?

- Does KV cache reduce the compute cost of length L sequence to linear?

Georgia Tech